NAME: B Sree Nikhil Reddy ID NO: 2520090058

SKILL WEEK 11

1. To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components.

Create a file:

Write the code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct
{
    int number1;
    int number2;
    char operation;
} Request;

typedef struct
{
    int result;
    int success;
    char message[100];
} Response;

int validate_request(Request *request)
{
    if (request->operation != '+' &&
        request->operation != '-' &&
        request->operation != '*' &&
        request->operation != '/')
    {
        return 0;
    }

    if (request->operation == '/' && request->number2 == 0)
        return 0;

    return 1;
}

Response process_request(Request *request)
{
    Response response;
    response.success = 1;
    strcpy(response.message, "Operation completed successfully.");

    switch (request->operation)
    {
        case '+':
            response.result = request->number1 + request->number2;
            break;

        case '-':
            response.result = request->number1 - request->number2;
            break;
```

```
        case '*':
            response.result = request->number1 * request->number2;
            break;

        case '/':
            response.result = request->number1 / request->number2;
            break;

        default:
            response.success = 0;
            strcpy(response.message, "Invalid operation.");
    }

    return response;
}

void display_response(Response *response)
{
    if (response->success)
        printf("Result: %d\n", response->result);
    else
        printf("Error: %s\n", response->message);
}

int main()
{
    Request request;
    Response response;

    printf("Module Integration Test\n");
    printf("-----------------------\n");

    printf("Enter first number: ");
    scanf("%d", &request.number1);

    printf("Enter operation (+ - * /): ");
    scanf(" %c", &request.operation);

    printf("Enter second number: ");
    scanf("%d", &request.number2);

    if (!validate_request(&request))
    {
        printf("Error: Invalid request.\n");
        return 1;
    }

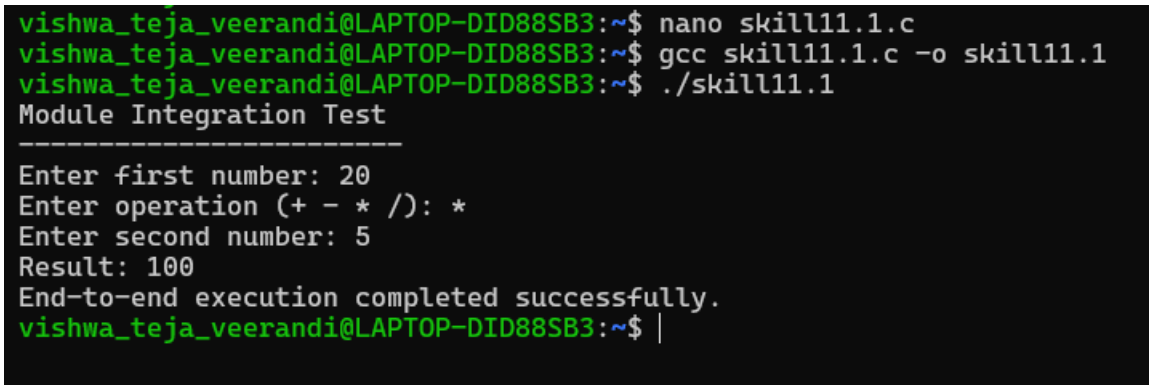
    response = process_request(&request);
```

```
    display_response(&response);

    printf("End-to-end execution completed successfully.\n");

    return 0;
}
```

OUTPUT:



2. To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios.

Create a file:

Write the code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>

void log_error(const char *message)
{
    FILE *log = fopen("skill11_errors.log", "a");

    if (log == NULL)
    {
        perror("Unable to open log file");
        return;
    }

    fprintf(log, "ERROR: %s\n", message);
    fclose(log);
}

int divide_numbers(int a, int b, int *result)
{
    if (b == 0)
    {
        log_error("Division by zero attempted.");
        return 0;
    }

    *result = a / b;
    return 1;
}

int main()
{
    int a, b, result;
    char operation;

    printf("Runtime Error Handling Test\n");
    printf("---------------------------\n");

    printf("Enter expression: ");
    if (scanf("%d %c %d", &a, &operation, &b) != 3)
    {
        printf("Error: Invalid syntax.\n");
        log_error("Invalid input syntax.");
        return 1;
    }

    if (operation != '/' &&
        operation != '+' &&
```

```
        operation != '-' &&
        operation != '*')
    {
        printf("Error: Unsupported operation.\n");
        log_error("Unsupported operation.");
        return 1;
    }

    if (operation == '/')
    {
        if (!divide_numbers(a, b, &result))
        {
            printf("Error: Cannot divide by zero.\n");
            printf("Program recovered gracefully.\n");
            return 1;
        }
    }
    else if (operation == '+')
    {
        result = a + b;
    }
    else if (operation == '-')
    {
        result = a - b;
    }
    else
    {
        result = a * b;
    }

    printf("Result: %d\n", result);
    printf("Execution completed successfully.\n");

    return 0;
}
```

OUTPUT:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill11.2.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc skill11.2.c -o skill11.2
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill11.2
Runtime Error Handling Test
---------------------------
Enter expression: 2/8
Result: 0
Execution completed successfully.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

3. To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings.

Create a file:

Write the code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    int size;
    int *numbers;
    char *message;

    printf("Valgrind Memory Test\n");
    printf("--------------------\n");

    printf("Enter number of integers: ");
    scanf("%d", &size);

    if (size <= 0)
    {
        printf("Invalid size.\n");
        return 1;
    }

    numbers = malloc(size * sizeof(int));

    if (numbers == NULL)
    {
        printf("Memory allocation failed.\n");
        return 1;
    }

    message = malloc(100);

    if (message == NULL)
    {
        free(numbers);
        printf("Memory allocation failed.\n");
        return 1;
    }

    strcpy(message, "Memory allocated successfully.");

    for (int i = 0; i < size; i++)
        numbers[i] = i + 1;

    printf("%s\n", message);

    printf("Values: ");

    for (int i = 0; i < size; i++)
```

```
        printf("%d ", numbers[i]);

    printf("\n");

    free(message);
    free(numbers);

    printf("All allocated memory released.\n");

    return 0;
}
```

OUTPUT:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill11.3.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc -g -O0 skill11.3.c -o skill11.3
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill11.3
Valgrind Memory Test
--------------------
Enter number of integers: 5
Memory allocated successfully.
Values: 1 2 3 4 5
All allocated memory released.
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```

4. To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports.

Create a file:

Write the code:

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    if (argc != 3)
    {
        printf("Usage: %s <number1> <number2>\n", argv[0]);
        return 1;
    }

    int a = atoi(argv[1]);
    int b = atoi(argv[2]);

    printf("Addition: %d\n", a + b);
    printf("Subtraction: %d\n", a - b);
    printf("Multiplication: %d\n", a * b);

    if (b != 0)
        printf("Division: %d\n", a / b);
    else
        printf("Division: Cannot divide by zero\n");

    return 0;
}
```

OUTPUT:

```
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ nano skill11.4.c
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ gcc skill11.4.c -o skill11.4
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$ ./skill11.4 20 5
Addition: 25
Subtraction: 15
Multiplication: 100
Division: 4
vishwa_teja_veerandi@LAPTOP-DID88SB3:~$
```